



VisionGuard **ULTRA**

Turning passive CCTV into an AI City Brain.

Detect → Decide → Dispatch.

Four people, one system



Abdul Rafay

Team Lead

AI pipeline, event engine,
voice, backend



Moiz

ML Engineer

Fire, weapon & violence
models



Areeba

Data Scientist

Data cleaning,
augmentation, evaluation



Aqib

Frontend & UX

React command center

THE PROBLEM

Cameras record. Nobody watches.

20M+

people in Karachi, with thousands of CCTV cameras

~20 min

before a screen operator's attention drops

After

CCTV gives evidence after the crime — not help during it

- One operator cannot watch 50 screens at once
- Help is late: someone must notice, then call 15 / 16 / 1122
- Many threat types: fights, weapons, fire, crowds, accidents, abandoned bags

OUR SOLUTION

An AI City Brain on existing cameras

VisionGuard watches every feed at the same time, finds emergencies in real time, scores the risk, saves tamper-proof evidence and routes the alert to the right department.

**Detect****Decide****Dispatch**

COVERAGE

12 event detectors

**Fire & smoke**

Growing flames, smoke plumes

**Fight**

Pose + violence model

**Weapon threat**

Weapon near a person

**Robbery**

Stop + fast rush

**Kidnapping**

Approach + struggle

**Crowd crush**

Density vs capacity

**Car accident**

Overlap + sudden stop

**Bike accident**

Rider lean + stop

**Loitering**

Same spot 5+ min

**Abandoned object**

Bag alone 60 s

**Vehicle obstruction**

Stopped 10+ s

**Audio emergency**

Gunshot, scream, blast

LIVE DEMO

What you are about to see

- 1 Command center**
All cameras analysed live
- 2 Fire video**
Fire & smoke → Fire Brigade 16
- 3 Fight / weapon video**
Pose skeletons + second check → Police 15
- 4 Risk score**
0–100, most dangerous first
- 5 Evidence locker**
Clip + SHA-256 fingerprint

RISK LEVELS**CRITICAL**

85–100

Dispatch suggested

HIGH

50–84

Team review

MEDIUM

25–49

Warning

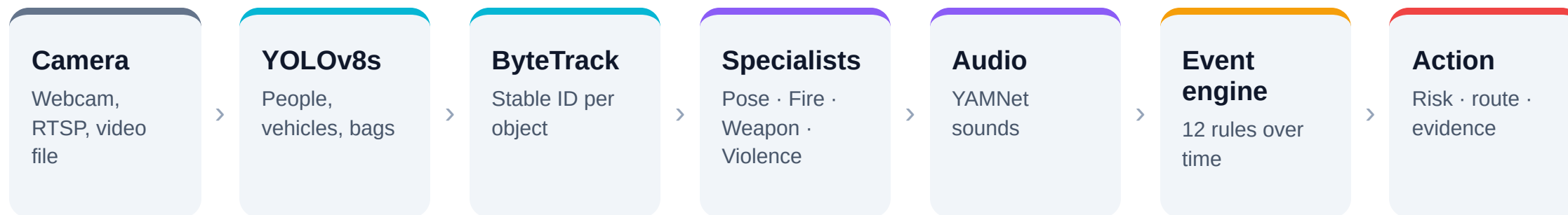
LOW

0–24

Ignored

HOW IT WORKS

From one frame to one alert



Time, not one frame

Events must last 1.5 s (weapons 0.5 s).
A grace period ignores one-frame flicker.



Two-stage check

Fast pose signal first, then the violence classifier confirms on the person crop.



Async pipeline

Display and heavy models run on separate threads, so the video never freezes.

MODELS

Transfer learning on our own data

MODEL	TYPE	DATA	INPUT	EPOCHS
Fire / Smoke v2	YOLOv8s detection	D-Fire + Roboflow (~16k images)	640	80
Weapon	YOLOv8s detection	CCTV weapon, knife, pistol (~7.1k)	640	60
Violence	YOLOv8s-cls	RWF-2000 + violence sets (~12k)	224	50
Pose · Detection · Audio	Pre-trained	YOLOv8s-pose · YOLOv8s (COCO) · YAMNet	—	—

Why YOLOv8?

Real-time on a 4 GB GPU. One framework for detection, pose and classification.

Fire v1 → v2

v1 used colour only (red clothes, sunsets = false alarms). v2 learns the shape and texture of fire and smoke.

Clean data, tested limits

16,027

D-Fire images scanned · 0 broken

7,116

clean weapon images · 5 duplicates removed

4,304

weather images made (rain, fog, shadow)

3

thresholds tuned from real false-alarm evidence

SETTING	BEFORE	AFTER	WHY
Crowd danger ratio	0.90	0.95	Crowd alarm fired too often
Event persistence	1.0 s	1.5 s	Duplicate alerts within 2 s
Detection confidence	0.30	0.35	False fights in dark frames

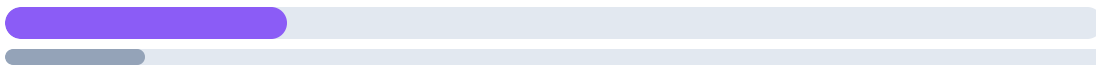
MEASURED RESULTS

Fire v2: 2× the old method

Precision 47.6% vs 25% (v1)



mAP@50 16.0% vs 8% (v1)



mAP@50-95 7.6% vs 3% (v1)



Recall 25.5% vs 35% (v1)



How to read this

- D-Fire is hard: many fires are tiny, far or hidden, and every box counts.
- We trade some recall for precision — fewer false alarms in a city.
- Alerts are decided over time: a fire missed in one frame is caught in the next.
- Weapon & violence evaluation: **in progress**. Target: mAP@50 $\geq$ 70%, Top-1 $\geq$ 85%.

PERFORMANCE

Async design: 43% faster on a small GPU

7 → 10

display FPS at 720p (+43%)

866 → 450

dropped frames (−48%)

42.7 → 29.9**S**

to process 300 frames (−30%)

Measured on an NVIDIA Quadro T1000 (4 GB). Next tuning: 480p input, every-2nd-frame AI, smart specialist scheduling, FP16 — target 22+ FPS.

Command center & evidence



Camera matrix

1×1, 2×2 and grid layouts. Location tree: city → sector → camera.



Live alerts

Pushed instantly over WebSocket, sorted by risk, colour-coded.



Voice control

"What is going on?" — Gemini turns speech into commands, with an offline fallback.



Evidence locker

5 s before + 3 s after each event, sealed with a SHA-256 fingerprint.



Heatmaps

Incident history shows where trouble happens most.



Human in the loop

AI suggests; the operator confirms or dismisses.

ROUTING

The right department, automatically

EVENT	DEPARTMENT	DIAL	PRIORITY
🔥 Fire	Fire Brigade	16	CRITICAL
🔫 Weapon · 💰 Robbery · 🚗 Kidnapping	Sindh Police / Rangers	15	CRITICAL
👤 Crowd crush	Disaster Management	1122	CRITICAL
🚗 Car / 🏍️ bike accident	Edhi / Rescue	115	CRITICAL / HIGH
👊 Fight · 🚒 Abandoned object	Police / Bomb Squad	15	HIGH
🚧 Vehicle obstruction	Traffic Police	1915	MEDIUM

TRUST

Behavioural, not biometric



No face recognition

We detect actions and objects — never who a person is. No face database.



Humans decide

Risk scores suggest; operators confirm before dispatch.



Tamper-proof

Every evidence file carries a SHA-256 hash. Any edit is detectable.



Access control

Login-protected dashboard; the city sets data retention.

What we will do next

KNOWN LIMITS

- Fire metrics still low on hard D-Fire set
- Weapon & violence reports in progress
- Small far weapons, night video
- Pixel rules depend on camera angle

NEXT STEPS

- More epochs, bigger model, Karachi data
- Finish evaluation of all 3 models
- Low-light augmentation, per-camera calibration
- City pilot · hospitals, schools, industry

Cameras are already there. VisionGuard gives them a brain.

Detect · Decide · Dispatch

Thank you — questions?

Abdul Rafay · Moiz · Areeba · Aqib · github.com/AbdulRafay70/Vision-Guard